

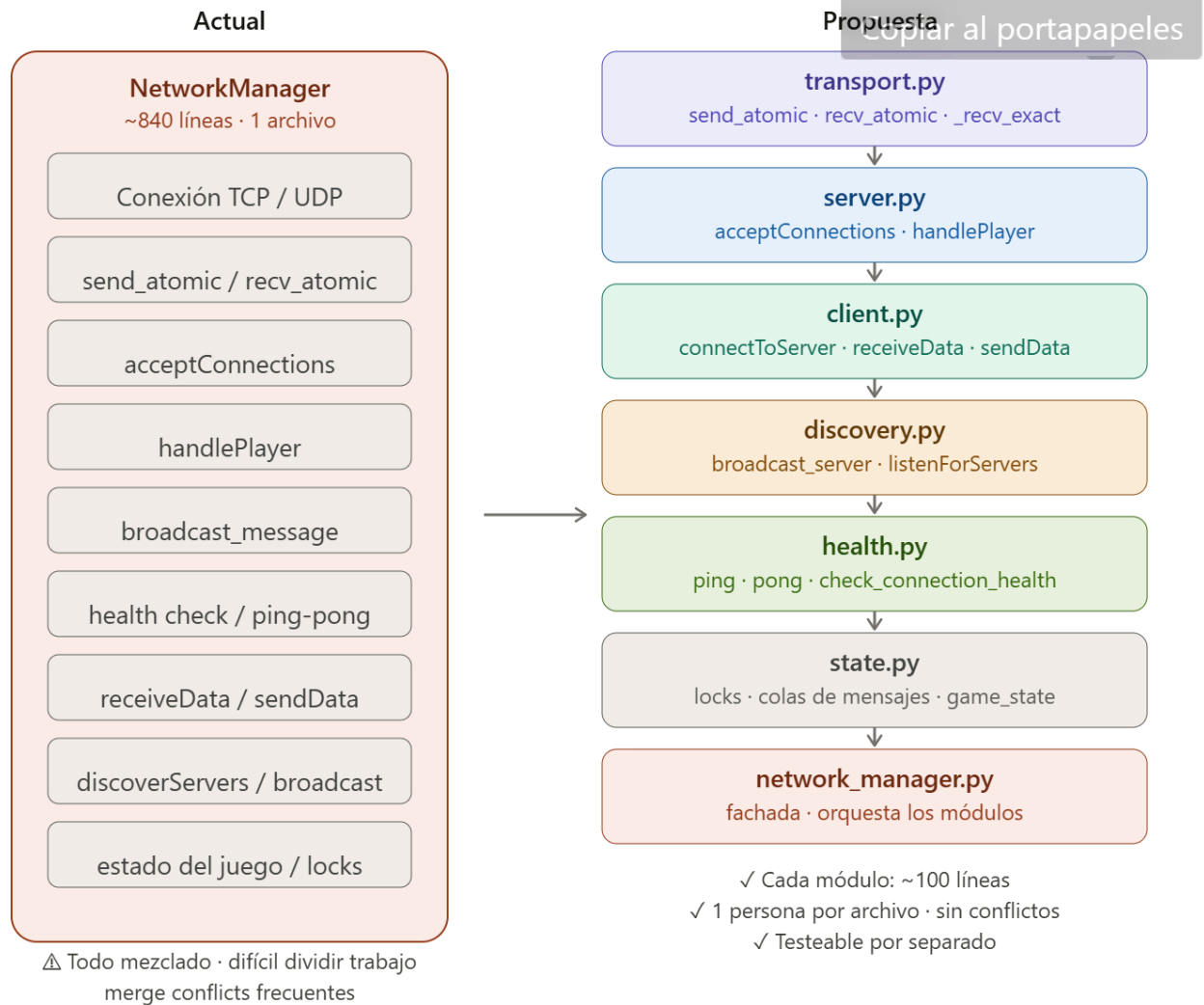
Parte 1: Iniciando el diseño FACADE y análisis del código base

Estudiamos el caso de que una refactorización grande sea posible sin dañar el código base restante

¿Es necesario refactorizar `network.py` ?

Sí, y es urgente. No porque el código esté roto, sino porque con 8 personas tocando el mismo archivo de ~840 líneas sin estructura clara, va a convertirse en un caos muy rápido. El código actual tiene el corazón correcto (atomic messaging, pickle, threading, health check), pero todo vive en una sola clase gigante que mezcla responsabilidades. Esto te va a dar problemas de merge conflicts, bugs difíciles de rastrear, y nuevos integrantes que no saben dónde tocar sin romper algo.

Te muestro primero la arquitectura actual vs la propuesta, y luego cómo dividir el trabajo entre tus 8 personas:



- Al final network_manager.py se terminó llamando simplemente manager.py
- También se añadió otros archivos
- config.py
- excepciones.py
- types.py
- constans.py

Ver DocStrings: explicaciones en clases y funciones

Diagnóstico de los problemas actuales

Antes de hablar del cómo, te explico exactamente qué está fallando en el código actual para que lo puedas justificar con tu equipo:

1. La clase hace demasiadas cosas a la vez. NetworkManager es simultáneamente servidor TCP, cliente TCP, emisor UDP, receptor de mensajes, gestor de estado del juego, y monitor de

salud. Cuando 8 personas trabajen en ella, van a pisar el trabajo de los demás constantemente.

2. El `__init__` tiene 30+ atributos sin organización. Hay variables comentadas, variables duplicadas (`received_data` y `receivedData`), y comentarios `# Nuevo...` que sugieren que se fue creciendo sin plan. Esto confunde a cualquier persona nueva.

3. El estado del juego vive mezclado con la red. Cosas como `moves_game` , `game_state` , `msgStartGame` deberían estar separadas para que el departamento de juego no dependa de cómo funciona la red internamente.

4. El manejo de errores es inconsistente. Algunos `except` capturan todo (`except:`) sin registrar qué pasó. Otros son muy específicos. Esto hace que los bugs sean difíciles de reproducir.

Cómo refactorizar (sin tocar nada más)

La idea central es el **patrón Facade**: `NetworkManager` sigue existiendo y se llama igual desde `main.py` y `ui2.py` , pero por dentro delega en módulos especializados. Cero cambios para los otros departamentos.

La estructura de carpetas quedaría así:

network/	
__init__.py	← exporta NetworkManager para que el resto del código
no cambie nada	
manager.py	← la fachada (NetworkManager), orquesta todo
transport.py	← envío/recepción atómica (send_atomic, recv_atomic)
server.py	← lógica del HOST (acceptConnections, handlePlayer)
client.py	← lógica del CLIENTE (connectToServer, receiveData)
discovery.py	← broadcast UDP y descubrimiento de salas
health.py	← ping-pong y detección de desconexiones
state.py	← colas de mensajes, locks, game_state

El `__init__.py` solo necesita esto:

```
# network/__init__.py
from .manager import NetworkManager
```

Y `manager.py` se convierte en la fachada delegada:

```

# network/manager.py
from .transport import Transport
from .server import GameServer
from .client import GameClient
from .discovery import Discovery
from .health import HealthMonitor
from .state import NetworkState

class NetworkManager:
    def __init__(self):
        self.state = NetworkState()          # variables compartidas con sus
locks
        self.transport = Transport()          # envío/recepción de bytes
        self.server = GameServer(self.state, self.transport)
        self.client = GameClient(self.state, self.transport)
        self.discovery = Discovery(self.state)
        self.health = HealthMonitor(self.state, self.transport)

    # Métodos públicos que ya usa el resto del código:
    def start_server(self, gameName, password, max_players, name_sala):
        return self.server.start(gameName, password, max_players, name_sala)

    def connectToServer(self, server):
        return self.client.connect(server)

    def sendData(self, data):
        return self.client.send(data)

    # ... etc. Solo wrappers de 1 línea.

```

División del trabajo para tus 8 integrantes

plan si se quiere ejecutar dividiendo requerimientos entre integrantes del departamenteo o grupo:

Persona	Módulo	Qué hace	Líneas aprox.
1	transport.py	send_atomic , recv_atomic , _recv_exact . El núcleo más crítico.	~80
2-3	server.py	start_server , acceptConnections , handlePlayer , reconexión. Dos personas	~200

Persona	Módulo	Qué hace	Líneas aprox.
		porque es el más complejo.	
4	client.py	connectToServer , receiveData , sendData .	~120
5	discovery.py	broadcast_server , listenForServers , discoverServers .	~80
6	health.py	start_health_check , check_connection_health , ping-pong.	~80
7	state.py	Todas las colas de mensajes, los locks, los getters thread-safe.	~100
8	manager.py + __init__.py	La fachada que integra todo. Esta persona también hace las pruebas de integración.	~80

Tres mejoras que vale la pena incluir al refactorizar

Estas no cambian nada externo pero elevan la calidad profesional significativamente:

Logging en lugar de print : Cambiar todos los `print(f"...")` por `logging.info(...)` . Esto permite activar/desactivar mensajes por módulo sin tocar el código, y es estándar en cualquier aplicación de red seria.

```
import logging
logger = logging.getLogger(__name__) # cada módulo tiene su propio logger

# En lugar de: print(f"Jugador {name} conectado")
logger.info("Jugador %s conectado (ID: %d)", name, player_id)
```

Constantes centralizadas: Los strings de tipo de mensaje como "BAJARSE" , "TOMAR_CARTA" , "PING" están hardcodedos en varios lugares. Un typo y el juego falla silenciosamente. Un archivo `constants.py` lo previene:

```
# network/constants.py
class MsgType:
    PING = "PING"
    PONG = "PONG"
    START_GAME = "START_GAME"
    BAJARSE = "BAJARSE"
    # ...
```

`connected_players` como lista de objetos en lugar de tuplas: Actualmente es `(conn, addr, name, id)`. Cuando alguien escribe `player[2]` en una parte y `player[3]` en otra, es imposible saber qué significa sin leer todo el contexto. Un dataclass simple lo resuelve:

```
from dataclasses import dataclass
import socket

@dataclass
class ConnectedPlayer:
    conn: socket.socket
    addr: tuple
    name: str
    player_id: int
```

Resumen ejecutivo

Sí refactoriza, y hazlo antes de que el equipo de 8 empiece a trabajar en paralelo, no después. El código actual funciona pero no escala humanamente. La refactorización que te propongo no toca nada de `Game`, `Round`, `Player`, `ui`, `ui2` ni `main`, solo reorganiza `network.py` en una carpeta con 7 archivos pequeños y un `__init__.py` que mantiene la interfaz pública exactamente igual. Para los otros 10 integrantes de los otros departamentos, nada cambia.

Sugerencias adicionales a considerar antes de integrar al `context_ia.md`:

1. **Manejo de excepciones estructurado:** Crear una clase `NetworkException` personalizada con códigos de error específicos (`TIMEOUT`, `DISCONNECTED`, `AUTH_FAILED`, etc.) para que el código sea más robusto.
2. **Configuration class:** Centralizar puertos, timeouts, reintentos, tamaños de buffer en una clase `NetworkConfig` en lugar de números mágicos dispersos.
3. **Documentación de tipos (Type hints):** Agregar anotaciones de tipo en todas las funciones (Python 3.10+) para que el IDE ayude a detectar errores tempranamente.
4. **Thread-safe queues:** Cambiar listas simples como `moves_game` y `incoming_messages` por `queue.Queue()` que es inherentemente thread-safe.
5. **Métodos de depuración:** Agregar un modo `DEBUG_MODE` que genera reportes de estado de red sin afectar el rendimiento.
6. **Reconexión automática cliente:** Implementar un mecanismo de reintentos exponencial cuando el cliente se desconecta accidentalmente.

7. **Tests unitarios:** Incluir templates de tests básicos que cada desarrollador pueda seguir para su módulo.
-

Parte 2: verificación y sugerencias

no implementado, solo a manera de revisión antes de la entrega del 08-0

Evaluación de la implementación

Veredicto general: **Superaste las expectativas base, con áreas específicas por mejorar**

No es un "más o menos". Es un trabajo sólido con decisiones inteligentes que van más allá del diseño original. Te lo desgloso con precisión.

Lo que hiciste muy bien

1. **Implementaste TODO lo propuesto y algo más.** El diseño original pedía 8 archivos, tú entregaste 11 (`config.py` , `exceptions.py` , `types.py` , `constants.py` extra). Eso demuestra criterio propio, no seguiste ciegamente el blueprint.

2. `server.py` **tiene decisiones de producción reales que no estaban en el diseño:**

- Limpias el estado anterior en `start()` con `connected_players.clear()` — previene jugadores fantasma entre partidas. Eso no estaba en el template.
- `self.state.update_last_activity(player_id, time.time())` inmediatamente al conectar — evita que el health monitor expulse a alguien antes de su primer PONG. Muy astuto.
- `_broadcast_players()` se llama tanto al conectar como al desconectar. Sincronización bidireccional correcta.

3. `health.py` **tiene una lógica que el diseño original omitió:**

```
if self.state.game_started:
    continue
```

No expulsar jugadores durante la partida por timeout es una decisión de gameplay crítica que el diseño original ignoró. La identificaste y la resolviste sola.

4. `client._process_message` **es completo y real**. El template era un esqueleto. Tu versión maneja 15+ tipos de mensaje con rutas específicas para cada uno, incluyendo el manejo especial de `PLAYER_ORDER` y `UPDATE_PLAYERS` a `incoming_messages`. Eso requirió leer el código de `ui2.py` y entender el contrato real.

5. `manager.py` **tiene la capa de compatibilidad correcta**:

```
@property
def connected_players(self):
    return [(p.conn, p.addr, p.name, p.player_id) for p in
self.state.get_connected_players()]
```

Mapear `ConnectedPlayer` a tuplas para no romper `ui.py` es exactamente la filosofía Facade bien aplicada. Sin ese detalle, el resto del proyecto habría fallado.

6. `discovery.py` **detecta la IP local real** con el truco del socket UDP a `10.255.255.255`. Eso no estaba en el diseño y es necesario para que el broadcast funcione en redes reales (no solo `127.0.0.1`).

Lo que está bien pero puede mejorar

1. `state.py` **tiene un alias redundante sin explicación**:

```
self.player_name = None
self.playerName = None # Alias usado en algunas partes del Server/Host
```

Dos variables para lo mismo sin documentar cuál es la canónica. Quien venga después va a usar una u otra al azar. La solución es un `@property` con un getter/setter que sincronice ambas, o eliminar una y documentar por qué existe.

2. `transport.py` **tiene un comportamiento silencioso en** `recv_atomic`:

```
except socket.timeout:
    raise # Permitir que suba al loop principal
```

Está bien técnicamente, pero el comentario solo dice "permitir que suba". No explica *por qué* este timeout es diferente al de `_recv_exact`. Un comentario como `# Timeout por`

inactividad (ningún byte recibido), el loop superior lo manejará con 'continue' sería la diferencia entre código que se entiende solo y código que se interpreta.

3. `health.py` no notifica a la UI cuando expulsa un jugador:

```
for player_id in disconnected:
    self.state.remove_connected_player(player_id)
```

Remueve al jugador del estado interno pero nunca llama a `_broadcast_players()` ni agrega un mensaje a `incoming_messages`. Si alguien se cae silenciosamente, los otros jugadores no se enteran. Falta agregar:

```
self._server._broadcast_players() # o un método en state
```

O mejor aún, un evento de desconexión en la cola.

4. `manager.get_exit_gameServer()` tiene un TODO sin resolver:

```
def get_exit_gameServer(self):
    # TODO: Implementar estado real si ui2.py lo demanda, por ahora lista
    vacía
    return []
```

Esto está bien como placeholder temporal, pero en un repositorio compartido un TODO sin fecha ni responsable asignado es una deuda técnica que nadie va a reclamar. Debería tener al menos: `# TODO [Dev5] [BLOCKER si ui2 lo usa]: Implementar cola de salidas.`

✗ Lo que falta y es importante

1. **No hay ningún archivo de tests.** El diseño propuso `network/tests/`. Con 8 personas trabajando en esto, sin tests, cualquier cambio en `transport.py` puede romper `server.py` sin que nadie lo sepa hasta ejecutar el juego completo.

2. `config.py` no se configura en el punto de entrada. `main.py` nunca hace:

```
logging.basicConfig(level=logging.INFO, ...)
```

Entonces todos los `logger.info(...)` de tus 8 módulos están escribiendo en el vacío. El logging existe pero no produce output sin la configuración inicial.

3. `discovery.py` **no maneja el caso de interfaz de red sin conexión**. Si el jugador tiene el WiFi apagado, el `sock.sendto(packet, ('<broadcast>', ...))` fallará en el `except Exception as e` y hará `time.sleep(1)`, pero el loop continúa. Debería marcar `running_broadcast = False` después de N fallos consecutivos.

Lo que le falta para ser excelente (En la documentacion)

- a) **Un "Cómo correr el módulo"** — alguien nuevo al proyecto no sabe si simplemente reemplaza `network.py` con la carpeta `network/` o si hay pasos adicionales.
 - b) **Tabla de cambios reales vs diseño original** — ¿qué decisiones tomaste diferente al `context_ia.md` y por qué? Eso es lo más valioso para el equipo.
 - c) **Sección "Decisiones de diseño"** — por ejemplo, explicar por qué `health.py` no expulsa durante la partida, o por qué `discovery.py` detecta la IP local con ese truco. Esas decisiones están en el código pero no en la documentación.
 - d) **Estado actual vs estado pendiente** — el TODO de `get_exit_gameServer` y la ausencia de tests deberían estar visibles en el documento para que el equipo sepa qué queda por hacer.
-

Resumen

Área	Nota
Arquitectura	✅ Excelente — superó el diseño
Compatibilidad con código existente	✅ Perfecta
Thread-safety	✅ Correcta
Decisiones de gameplay en red	✅ Muy buenas (health durante partida)
Logging	⚠️ Implementado pero sin activar en main.py
Variables duplicadas en state.py	⚠️ playerName / player_name
Tests	❌ Ausentes
Documentación técnica	⚠️ Buena como reporte, incompleta como doc de proyecto
TODOs documentados	⚠️ Existen pero sin responsable ni urgencia